Alg2 Report

Paul Molloy

November 6, 2025

1 Task 1

The traversal starts at the root of the suffix tree and for each character of the input string it attempts to follow the edge of the tree to find a match. Once a matching child node is found, the algorithm compares the characters on the edge labels with the characters in the input string. If the edge matches the traversal continues to the next node and the next character, process repeats itself until a match is found.

2 Task 2

To find all occurrences we make use of a function from task 1 to find the string in a given text. If the function from task 1 returns a match, then we begin to look for more occurrences from the node beneath that edge. We do this by counting the amount of leaf nodes present as they indicate an occurrence of the same string. The section of the tree is searched recursively from left to right and must go down until there are no nodes left. To do this, whenever a node has no children, the algorithm checks for a sibling.

3 Task 3

The longest repeated suffix is indicated on the tree by looking for the deepest branch node that has at least two children. Recursion is used to traverse the entire tree and a depth variable is stored to check the maximum depth, each time this is updated the positions of the two children are also recorded. By the end of execution the positions stored will be the occurrences of the repeated substring.

4 Task 4

To find the longest common substring, first we build a suffix tree by concatenating the two texts, separated with a '#'. The algorithm to fins this is similar to task three but we need to find a node that has children from both texts. The point at which '#' occurs is a split point and to check if a certain substring belongs to a text, we check if the edge label is greater than or less than the split point. A valid candidate has edges that are less and more than the split point. We consider this to be valid, then compare the parents nodes depth against the maximum.